

5강. React Router(라우터)



React

리액트 Router

- Router

****리액트 라우터(React Router)****는 리액트 앱에서 ****페이지 이동(Routing)****을 구현하기 위해 가장 널리 사용되는 표준 라이브러리입니다.

리액트는 기본적으로 **SPA(Single Page Application)** 방식이기 때문에, 서버에서 매번 새로운 HTML을 받아오는 대신 하나의 페이지 안에서 필요한 컴포넌트만 갈아끼우는 방식으로 작동합니다.

이때 사용자가 접속한 URL 주소에 따라 어떤 컴포넌트를 보여줄지 결정하는 역할을 리액트 라우터가 담당합니다.



리엑트 Router

- 주요 컴포넌트 및 기능

구성 요소	설명
BrowserRouter	라우터의 최상위 컴포넌트입니다. HTML5 History API를 사용해 UI와 URL을 동기화합니다.
Routes & Route	경로를 정의합니다. path 속성에는 URL을, element에는 해당 주소에서 보여줄 컴포넌트를 넣습니다.
Link	HTML의 <a> 태그와 비슷하지만, 페이지 새로고침을 막고 라우터를 통해 이동하게 합니다.
useNavigate	특정 이벤트(버튼 클릭 등)가 발생했을 때 프로그래밍 방식으로 페이지를 이동시킬 때 쓰는 Hook입니다.



리엑트 Router

- react-router-dom 라이브러리 설치

```
npm install react-router-dom
```

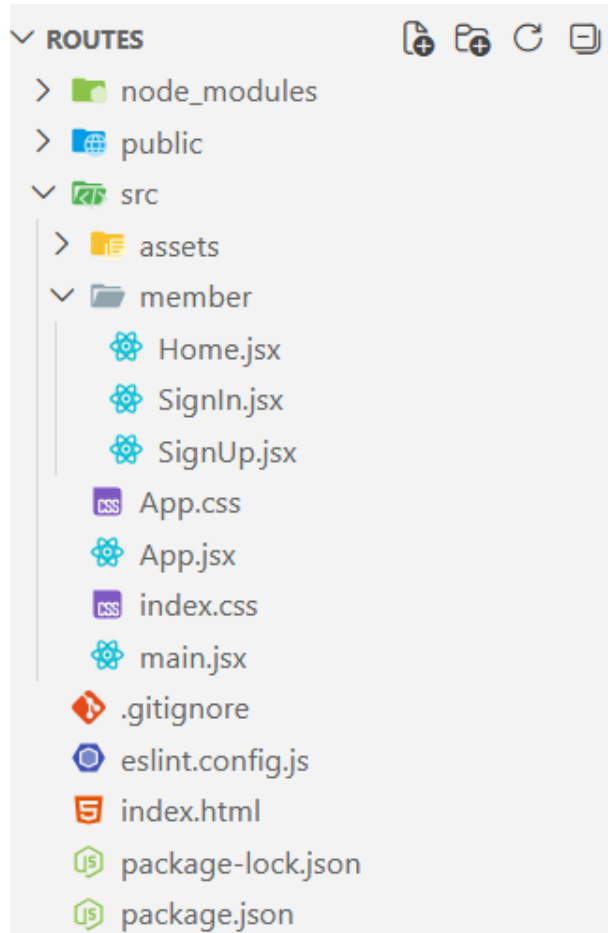
package.json

```
"dependencies": {  
  "react": "^19.2.6",  
  "react-dom": "^19.2.6",  
  "react-router-dom": "^7.16.0"  
},
```



리엑트 Router

- routes Project 생성

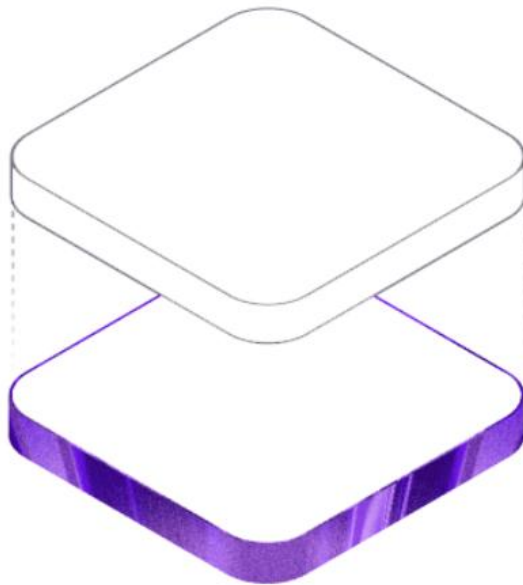


리엑트 Router

[Home](#) [로그인](#) [회원가입](#)

라우터를 테스트합니다.

[상단의 메뉴를 클릭해 보세요.]



리엑트 Router

[Home](#) [로그인](#) [회원가입](#)

로그인



리엑트 Router

[Home](#) [로그인](#) [회원가입](#)

회원가입

이름

직업

성별 ☐ 남자 ☐ 여자

자기소개



리엑트 Router

▪ Header 영역

```
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom'
import Home from './member/Home'
import SignIn from './member/SignIn'
import SignUp from './member/SignUp'

function App() {

  return (
    <>
      <section className='app'>
        <BrowserRouter>
          <div className='header'>
            <Link to="/">Home</Link>
            <Link to="/sign-in">로그인</Link>
            <Link to="/sign-up">회원가입</Link>
          </div>
        </BrowserRouter>
      </section>
    </>
  )
}
```



리엑트 Router

- Main 영역

```
    <div className='contents'>
      <Routes>
        <Route path='/' element={<Home />} />
        <Route path="/sign-in" element={<SignIn />} />
        <Route path='/sign-up' element={<SignUp />} />
      </Routes>
    </div>
  </BrowserRouter>
</section>
</>
)
}

export default App
```



리엑트 Router

▪ App.jsx

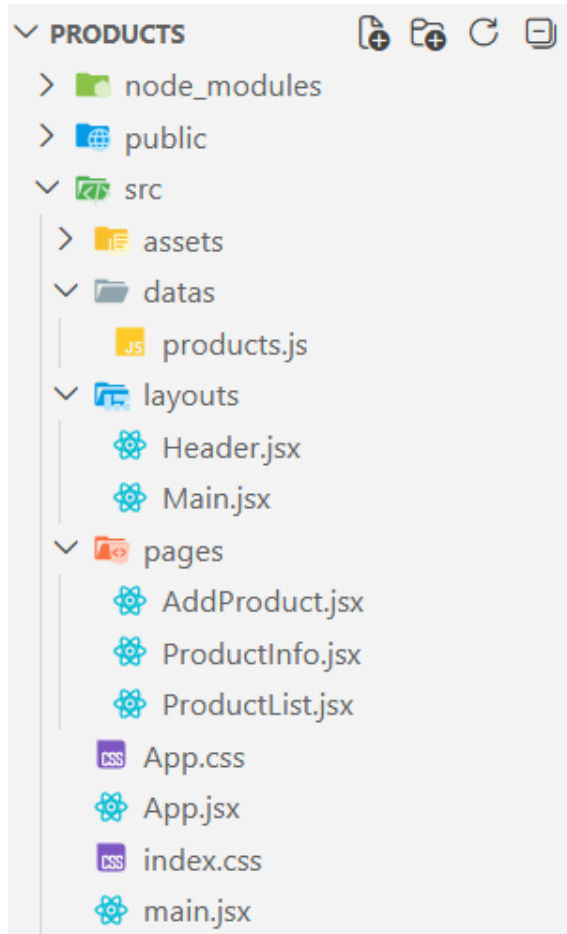
```
▼ <body>
  ▼ <div id="root"> flex
    ▼ <section class="app">
      ▼ <div class="header">
        ...    <a href="/" data-discover="true">Home</a> == $0
              <a href="/sign-in" data-discover="true">로그인</a>
              <a href="/sign-up" data-discover="true">회원가입</a>
            </div>
            ▶ <div class="contents"> ... </div>
          </section>
        </div>
```

Link가 <a>태그로 렌더링 됨



상품 관리 앱 라우팅

■ 상품 관리 프로젝트 구조



상품 관리 앱 라우팅

▪ Header.jsx

```
import { Link } from "react-router-dom"

const Header = () => {
  return (
    <header className="header">
      <Link to="/">Home</Link>
    </header>
  )
}

export default Header
```



상품 관리 앱 라우팅

- Main.jsx

```
const Main = () => {  
  return (  
    <main>  
      <h1>컴퓨터 기기 쇼핑몰</h1>  
    </main>  
  )  
}  
  
export default Main
```



상품 관리 앱 라우팅

▪ App.jsx

```
import { BrowserRouter } from 'react-router-dom'

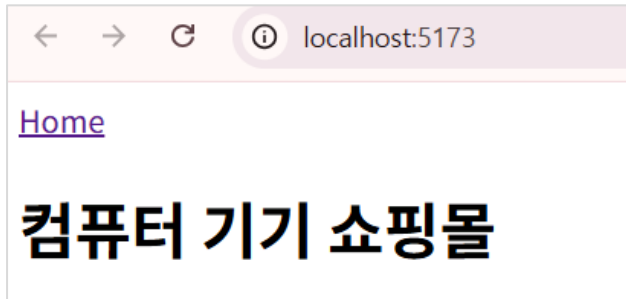
function App() {

  return (
    <>
      <div className="App">
        <BrowserRouter>
          <Header />
          <Main />
        </BrowserRouter>
      </div>
    </>
  )
}
```



상품 관리 앱 라우팅

▪ App.jsx



```
<body>
  <div id="root">
    <div class="App">
      <header class="header">
        <a href="/" data-discover="true">Home</a> == $0
      </header>
      <main>
        <h1>컴퓨터 기기 쇼핑몰</h1>
      </main>
    </div>
  </div>
  <script type="module" src="/src/main.jsx?t=1773993773892">
</body>
```

Link가 <a>태그로
렌더링 됨



상품 관리 앱 라우팅

▪ 경로 정의 – Routes, Route

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
```

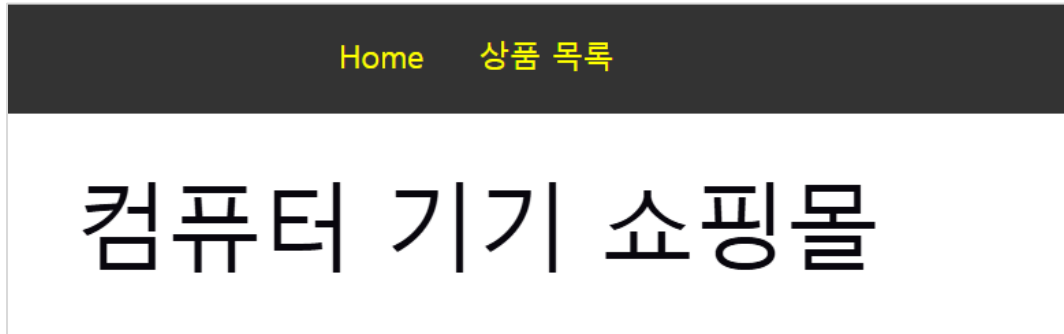
```
return (  
  <>  
    <div className="app">  
      <BrowserRouter>  
        <Header />  
        { /* <Main /> */ }  
        <Routes>  
          <Route path="/" element={<Main />} />  
        </Routes>  
      </BrowserRouter>  
    </div>  
  </>  
)
```

<Route>는 <Routes>
안에 있어야함



상품 관리 앱 라우팅

- Home 화면



상품 관리 앱 라우팅

- 상품 목록 화면



상품 관리 앱 라우팅

- 상품 목록 페이지 이동 – Header.jsx

```
const Header = () => {  
  return (  
    <header className="header">  
      <Link to="/">Home</Link>  
      <Link to="/products">상품 목록</Link>  
    </header>  
  )  
}
```



상품 관리 앱 라우팅

▪ App.css

```
/* Header 스타일 */  
.header{  
  padding: 20px;  
  background-color: #333;  
}  
.header a{  
  margin: 0 16px;  
  color: yellow;  
  text-decoration: none;  
}  
.header a:hover{  
  color: #eee;  
}  
  
/* Main 스타일 */  
.main{padding: 20px;}  
  
/* 상품 목록 스타일 */  
.product-list{padding: 20px;}
```



상품 관리 앱 라우팅

- 객체 데이터 리스트 – datas/products.js

```
export const products = [  
  {  
    id: 1,  
    name: '모니터',  
    price: 130000,  
    description: '22인치 최신 모니터입니다.',  
  },  
  {  
    id: 2,  
    name: '마우스',  
    price: 30000,  
    description: '무선 마우스입니다.',  
  },  
  {  
    id: 3,  
    name: '키보드',  
    price: 50000,  
    description: '무선 키보드입니다.',  
  },  
]
```



상품 관리 앱 라우팅

■ 객체 데이터 리스트 – ProductList.jsx

```
import {products} from "../datas/products"
import { Link} from "react-router-dom";

const ProductList = () => {

  return(
    <div className="product-list">
      <h2>상품 리스트</h2>
      <ul>
        {products.map((product) => (
          <li key={product.id}>
            <h3>{product.name}</h3>
            <p>가격: {product.price}원</p>
            <p>설명: {product.description}</p>
          </li>
        ))}
      </ul>
    </div>
  )
}
```



상품 관리 앱 라우팅

- 상품 상세 보기 – ProductInfo.jsx

```
const ProductInfo = () => {  
  
  return (  
    <section className="product-info">  
      <h2>상품 정보</h2>  
    </section>  
  )  
}  
  
export default ProductInfo
```



상품 관리 앱 라우팅

- 상품 상세 보기 링크 추가 – ProductList.jsx

```
<h2>상품 리스트</h2>
{products.map((product) => (
  <div key={product.id}>
    <h3>
      <Link to={` /products/${product.id}`}>{product.name}</Link>
    </h3>
    <p>가격: {product.price}원</p>
```



상품 관리 앱 라우팅

- 상품 상세 보기 경로 - App.jsx

```
<BrowserRouter>
  <Header />
  {/* <Main /> */}
  <Routes>
    <Route path="/" element={<Main />} />
    <Route path="/products" element={<ProductList />} />
    {/* URL 파라미터를 포함한 경로 설정 */}
    {/* :id는 URL에서 동적으로 변하는 부분을 나타냅니다. */}
    {/* 예를 들어, /products/1, /products/2 등 다양한 상품 ID에 대응. */}
    <Route path="/products/:id" element={<ProductInfo />} />
  </Routes>
</BrowserRouter>
```



상품 관리 앱 라우팅

- 상품 상세 보기



상품 관리 앱 라우팅

- useParams()로 id 받아오기

```
import { useParams } from 'react-router-dom'

const ProductInfo = () => {
  // URL 파라미터에서 id 값을 추출
  // useParams 훅을 사용하여 URL에서 id 값을 가져옵니다.
  // 예를 들어, URL이 /products/1 이면 id는 1이 됩니다.
  const { id } = useParams()

  return (
    <section className="product-info">
      <h2>상품 정보</h2>
      <p>상품 ID: {id}</p>
    </section>
  )
}
```



상품 관리 앱 라우팅

▪ 상품 상세 보기

```
import { useParams, useNavigate } from "react-router-dom";
import { products } from "../datas/products"

const ProductInfo = () => {
  // URL 파라미터에서 id 값을 추출
  const {id} = useParams()
  // 페이지 이동 hook
  const navigate = useNavigate();

  // products 리스트에 id와 일치하는 상품 찾기
  const product = products.find(product =>
    product.id === parseInt(id))

  // 페이지 이동 핸들러
  const doClick = () => {
    navigate('/products');
  }
}
```



상품 관리 앱 라우팅

▪ 상품 상세 보기

```
return(  
  <div className="product-info">  
    <h3>상품 ID: {id}</h3>  
    /* <p>가격: {products[id-1].price}</p>  
    <p>설명: {products[id-1].description}</p> */  
    <p>이름: {product.name}</p>  
    <p>가격: {product.price}</p>  
    <p>설명: {product.description}</p>  
  
    /* <button onClick={() => {navigate('/products')}}>목록보기</button> */  
    <button onClick={doClick}>목록보기</button>  
  </div>  
)  
}
```



상품 관리 앱 라우팅

■ 상품 등록

[Home](#) [상품 목록](#) [상품 등록](#)

localhost:5173 내용:
상품이 등록되었습니다.

확인

상품 등록

상품명

가격

설명

128GB USB 메모리 입니다.

등록

취소

```
상품 추가: AddProduct.jsx:39
▼ {name: 'USB 메모리', price: '13000', description: '128GB USB 메모리 입니다.'} ⓘ
  description: "128GB USB 메모리 입니다."
  name: "USB 메모리"
  price: "13000"
```



상품 관리 앱 라우팅

▪ App.jsx

```
import AddProduct from './pages/AddProduct'

function App() {

  return (
    <>
      <section className="app">
        <BrowserRouter>
          <Header />
          <Routes>
            <Route path="/" element={<Main />} />
            <Route path="/products" element={<ProductList />} />
            <Route path="/products/:id" element={<ProductInfo />} />
            <Route path="/add-product" element={<AddProduct />} />
          </Routes>
        </BrowserRouter>
      </section>
    </>
  )
}
```



상품 관리 앱 라우팅

▪ Header.jsx

```
import { Link } from "react-router-dom"

const Header = () => {
  return (
    <header>
      <header className="header">
        <Link to="/">Home</Link>
        <Link to="/products">상품 목록</Link>
        <Link to="/add-product">상품 등록</Link>
      </header>
    </header>
  )
}

export default Header
```



상품 관리 앱 라우팅

- 상품 목록 페이지에서 등록 버튼 생성

키보드

가격: 40000원

설명: 무선 키보드입니다.

상품 등록하기

ProductList.jsx

```
<div className="btn-add">  
  <Link to="/add-product">  
    <button type="button">상품 등록하기</button>  
  </Link>  
</div>  
</section>
```



상품 관리 앱 라우팅

▪ 상품 등록 – AddProduct.jsx

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";

const AddProduct = () => {
  // 폼 데이터 상태 관리
  const [formData, setFormData] = useState({
    name: '',
    price: '',
    description: ''
  })

  const navigate = useNavigate();

  // 입력 필드 변경 핸들러
  const handleChange = (e) => {
    const { name, value } = e.target

    // 변경된 name 값만 업데이트
    setFormData({...formData, [name]: value})
  }
}
```



상품 관리 앱 라우팅

▪ 상품 등록 – AddProduct.jsx

```
// 폼 전송 핸들러
const handleSubmit = (e) => {
  e.preventDefault();

  // 입력값 검증(유효성 검사)
  if(!formData.name.trim() || !formData.price ||
    !formData.description.trim()){
    alert("모든 필드를 입력해주세요");
    return; //즉시 종료
  }

  // 가격 필드 검증
  if(isNaN(formData.price) || formData.price <= 0){
    alert("가격은 0보다 큰 숫자를 입력해주세요");
    return;
  }

  console.log('상품 추가:', formData);
  alert("상품이 등록되었습니다.")
  navigate("/products"); //상품 목록 페이지로 이동
}
```



상품 관리 앱 라우팅

■ 상품 등록 – AddProduct.jsx

```
// 입력 취소 버튼
const handleCancel = () => {
  //입력 필드 초기화
  setFormData({
    name: '',
    price: '',
    description: ''
  })
}

return(
  <div className="add-product">
    <h2>상품 등록</h2>
    <form onSubmit={handleSubmit} className="add-form">
      <div>
        <label htmlFor="name">상품명 </label>
        <input
          type="text"
          id="name"
          name="name"
          value={formData.name}
          onChange={handleChange}
          placeholder="상품명을 입력하세요"
        />
      </div>
    </form>
  </div>
)
```



상품 관리 앱 라우팅

▪ 상품 등록 – AddProduct.jsx

```
</div>
<div>
  <label htmlFor="price">가격 </label>
  <input
    type="number"
    id="price"
    name="price"
    value={formData.price}
    onChange={handleChange}
    placeholder="가격을 입력하세요"
  />
</div>
<div>
  <label htmlFor="description">설명 </label>
  <textarea
    id="description"
    name="description"
    value={formData.description}
    onChange={handleChange}
```



상품 관리 앱 라우팅

■ 상품 등록 – AddProduct.jsx

```
        rows={5}
        cols={30}
        placeholder="상품 설명을 입력하세요"
      />
    </div>
    <div>
      <button type="submit">등록</button>
      <button type="button" onClick={handleCancel}>취소</button>
    </div>
  </form>
</div>
)
}

export default AddProduct;
```



상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기



상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – App.jsx

```
function App() {  
  // 로그인 상태 관리  
  const [isLoggedIn, setIsLoggedIn] = useState(false)  
  // 로그인한 사용자 ID 관리  
  const [userId, setUserId] = useState('')  
  
  // 로그인 핸들러  
  const handleLogin = (userId) => {  
    setIsLoggedIn(true) // 로그인 성공 시 상태 업데이트  
    setUserId(userId) // 로그인한 사용자 ID 저장  
  }  
  
  // 로그아웃 핸들러  
  const handleLogout = () => {  
    setIsLoggedIn(false)  
    setUserId('')  
  }  
}
```



상품 관리 앱 라우팅

로그인 / 로그아웃 메뉴 만들기 - App.jsx

```
return (  
  <>  
    <div className="app">  
      <BrowserRouter>  
        <Header isLoggedIn={isLoggedIn} userId={userId} onLogout={handleLogout} />  
        { /* <Main /> */ }  
        <Routes>  
          <Route path="/" element={<Main />} />  
          <Route path="/products" element={<ProductList />} />  
          <Route path="/products/:id" element={<ProductInfo />} />  
          <Route path="/add-product" element={<AddProduct />} />  
          <Route path="/signin" element={<SignIn onLogin={handleLogin} />} />  
        </Routes>  
      </BrowserRouter>  
    </div>  
  </>  
)  
}
```



상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – Header.jsx

```
import { Link } from "react-router-dom"
import { useNavigate } from "react-router-dom"

const Header = ({ isLoggedIn, userId, onLogout }) => {
  const navigate = useNavigate() // 페이지 이동을 위한 훅

  return (
    <header className="header">
      <Link to="/">Home</Link>
      <Link to="/products">상품 목록</Link>
      <Link to="/add-product">상품 등록</Link>
      {isLoggedIn ? (
        <div className="header-user">
          <span>{userId}님</span>

```



상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – Header.jsx

```
    <button
      type="button"
      className="logout-btn"
      onClick={() => { onLogout(); navigate('/'); }}
    >
      로그아웃
    </button>
  </div>
) : (
  <Link to="/signin">로그인</Link>
)}
</header>
)
}
```



상품 관리 앱 라우팅

- 로그인 / 로그아웃 메뉴 만들기 – App.css

```
.header-user {  
  display: inline-flex;  
  align-items: center;  
  gap: 10px;  
  color: white; #fff;  
  margin-left: 12px;  
}  
  
.logout-btn {  
  padding: 6px 12px;  
  border: 1px solid orange; #ff9f1c;  
  border-radius: 6px;  
  background-color: transparent;  
  color: orange; #ff9f1c;  
  cursor: pointer;  
}  
  
.logout-btn:hover {  
  background-color: orange; #ff9f1c;  
  color: black; #333;  
}
```



상품 관리 앱 라우팅

로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
const SignIn = ({ onLogin }) => {  
  const navigate = useNavigate(); // 페이지 이동을 위한 훅  
  // 1. 입력 데이터를 객체로 통합  
  const [loginData, setLoginData] = useState(  
    { userId: '', password: '' }  
  );  
  // 로그인 결과 상태 (성공, 실패, 또는 null)  
  const [result, setResult] = useState(null);  
  
  // 2. 통합 핸들러 (name 속성 활용)  
  const handleChange = (e) => {  
    const { name, value } = e.target;  
    console.log(e);  
  
    setLoginData({ ...loginData, [name]: value });  
    setResult(null); // 입력 시 결과 메시지 숨김  
  };
```



상품 관리 앱 라우팅

로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
// 3. 제출 핸들러
const handleSubmit = (e) => {
  e.preventDefault();
  const { userId, password } = loginData;

  //find 메서드로 사용자 데이터에서 첫번째 일치하는 항목 찾기
  const matched = users.find(
    (user) => user.userId === userId && user.password === password
  );

  // 로그인 성공 여부에 따라 결과 상태 업데이트
  if (matched) {
    setResult('success');
    onLogin(userId); // App에 로그인 정보 전달
    navigate('/');
  } else {
    setResult('fail');
  }
};
```



상품 관리 앱 라우팅

로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
return (  
  <div className='sign-in'>  
    <h2>로그인</h2>  
    <form onSubmit={handleSubmit}>  
      <p>  
        <input  
          name="userId" // name 추가  
          type="text"  
          placeholder="ID 입력"  
          value={loginData.userId}  
          onChange={handleChange}  
        />  
      </p>  
      <p>  
        <input  
          name="password" // name 추가  
          type="password"  
          placeholder="패 스 워 드  입 력 "  
          value={loginData.password}  
          onChange={handleChange}  
        />  
      </p>  
    </form>  
  </div>  
)
```



상품 관리 앱 라우팅

로그인 / 로그아웃 메뉴 만들기 – SignIn.jsx

```
    <button type="submit">로그인</button>
  </form>

  {/* 결과 메시지 부분 */}
  {result === 'success' && (
    <p style={{ color: 'green' }}>환영합니다, {loginData.userId}님.</p>
  )}
  {result === 'fail' && (
    <p style={{ color: 'red' }}>아이디 또는 비밀번호가 올바르지 않습니다.</p>
  )}
</div>
);
};

export default SignIn;
```



실습 문제

- 문제

리엑트 라우터를 이용하여 3개의 페이지를 만들고, 메뉴 클릭시 페이지가 이동하도록 만드세요

Home
About
Board



실습 문제

- 문제

동적 라우팅을 구현하세요

```
/board/1
```

```
/board/2
```

